

Applied Optimization

Exercise 3 - Convex Optimization Problems

September 18, 2025

What to hand in

A .zip compressed file renamed to `Exercise $n$ -Group.zip` where n is the number of the current exercise sheet. It should contain:

- Hand in **only** the files you changed (headers and source). It is up to you to make sure that all files that you have changed are in the zip.
- A `readme.txt` file containing a description on how you solved each exercise (use the same numbers and titles) and the encountered problems. Indicate what fraction of the total workload each project member contributed.
Failure to do so will result in you losing a point.
- Other files that are required by your `readme.txt` file. For example, if you mention some screenshot images in `readme.txt`, these images need to be submitted too.
- For the theory exercise, put `TheoryExercise.pdf` with your solutions in the same .zip you submit for the code.
- Submit your solutions to ILIAS before the submission deadline. Late submissions will receive 0 points!
- **Important:** Your submission MUST compile without any issue in order to obtain a passing grade.

QCQP to SOCP (0.5 pt)

1. Show that a **quadratic** constraint of the form

$$x^T A^T A x + b^T x + c \leq 0$$

is equivalent to the **second order cone** constraint

$$\left\| \begin{array}{c} \frac{1+(b^T x + c)}{2} \\ Ax \end{array} \right\|_2 \leq \frac{1 - (b^T x + c)}{2}$$

2. Convert the following QCQP into SOCP:

$$\begin{aligned} &\text{minimize} && x_1^2 + 4x_1x_2 + 4x_2^2 \\ &\text{subject to} && 9x_1^2 + 16x_2^2 \leq 25 \\ &&& x_1 - x_2 = 1 \end{aligned}$$

Linear Programming

Transform (0.5 pt)

For the following optimization problem

$$\begin{aligned} &\text{minimize} && \|(2x_1 + 3x_2, -3x_1)^T\|_\infty \\ &\text{subject to} && |x_1 - 2x_2| \leq 3 \end{aligned}$$

(1) Express the problem as a linear program. (2) Convert the LP so that all variables are in $\mathbb{R}_+$ and there is no other inequality constraints that **each individual variable** is ≥ 0 .

Transform general LP to standard form (Bonus 0.5 pt)

A general linear program has the form

$$\begin{aligned} &\text{minimize} && c^T x + d \\ &\text{subject to} && Gx \preceq h \\ &&& Ax = b \end{aligned}$$

where $G \in \mathbb{R}^{m \times n}$ and $A \in \mathbb{R}^{p \times n}$. Transform the general LP to its standard form:

$$\begin{aligned} &\text{minimize} && p^T x' \\ &\text{subject to} && Bx' = e \\ &&& x' \geq 0 \end{aligned}$$

Explain in detail the relation between the feasible sets, the optimal solutions, and the optimal values of the standard form LP and the original LP.

Programming Exercise: Mass Spring System (1 pt)

Consider a system of m by n springs connecting nodes as illustrated in Figure 1. The coordinate $\mathbf{x}$ of a node indexed a at grid point (i, j) , where $a \in (0, \dots, (m+1) * (n+1) - 1)$, $i \in (0, \dots, m)$ and $j \in (0, \dots, n)$, is not fixed in $\mathbb{R}^2$. Each edge of the grid has an elastic constant k and each node is subject to an elastic force proportional to the distance between the nodes. Let $\mathbf{x}_a$ and $\mathbf{x}_b$ denote the positions of nodes a and b respectively connected by an edge with elastic constant $k_{a,b}$. Compute the total potential energy stored in the system for a given set of positions under the following modeling assumptions: (1) ideal springs without length and (2) springs with length. Denote with $\|\mathbf{x}_a - \mathbf{x}_b\|$ the euclidean distance between $\mathbf{x}_a$ and $\mathbf{x}_b$.

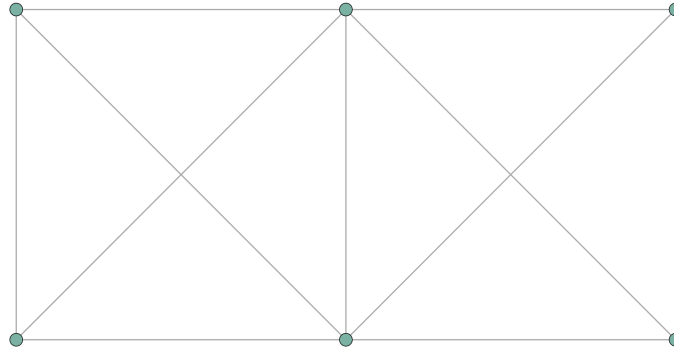


Figure 1: Example: 2×1 grid

1. Springs without length In this case magnitude of the force between this nodes will be given by:

$$F_{a,b} = k_{a,b} \|\mathbf{x}_a - \mathbf{x}_b\|,$$

and the potential energy between nodes

$$E_{a,b} = \frac{1}{2} k_{a,b} \|\mathbf{x}_a - \mathbf{x}_b\|^2,$$

Thus the total energy in the system is the sum of the energies of the individual edges $e(a,b)$

$$E = \sum_{e(a,b)} E_{a,b}$$

2. Springs with length Now let the rest length of the spring be $l_{i,j}$. The potential energy between nodes is

$$\hat{E}_{a,b} = \frac{1}{2} k_{a,b} (\|\mathbf{x}_a - \mathbf{x}_b\|^2 - l_{a,b}^2)^2,$$

Thus the total energy in the system is the sum of the energies of the individual edges $e(i,j)$

$$\hat{E} = \sum_{e(a,b)} \hat{E}_{a,b}$$

In this exercise, you are requested to evaluate the function value, the gradient and the hessian of the mass spring system at a certain status. The system can be viewed as a combination of elements, with each spring being an element. In `SpringElement2D.hh`

and `SpringElement2DWithLength.hh`, with the energy functions $E_{a,b}$ and $\hat{E}_{a,b}$ given above, the functions `eval_f(...)`, `eval_gradient(...)` and `eval_hessian(...)` should be implemented accordingly. Then the local data of each element is assembled into the `MassSpringProblem`. The total energy is simply the accumulation of every spring's function value. Then local gradient and hessian should be assembled in the global gradient and hessian according to the global indices of the end vertices of every spring. Figure 2 shows how the local gradient vector of the spring marked as red is assemble to the global gradient vector.

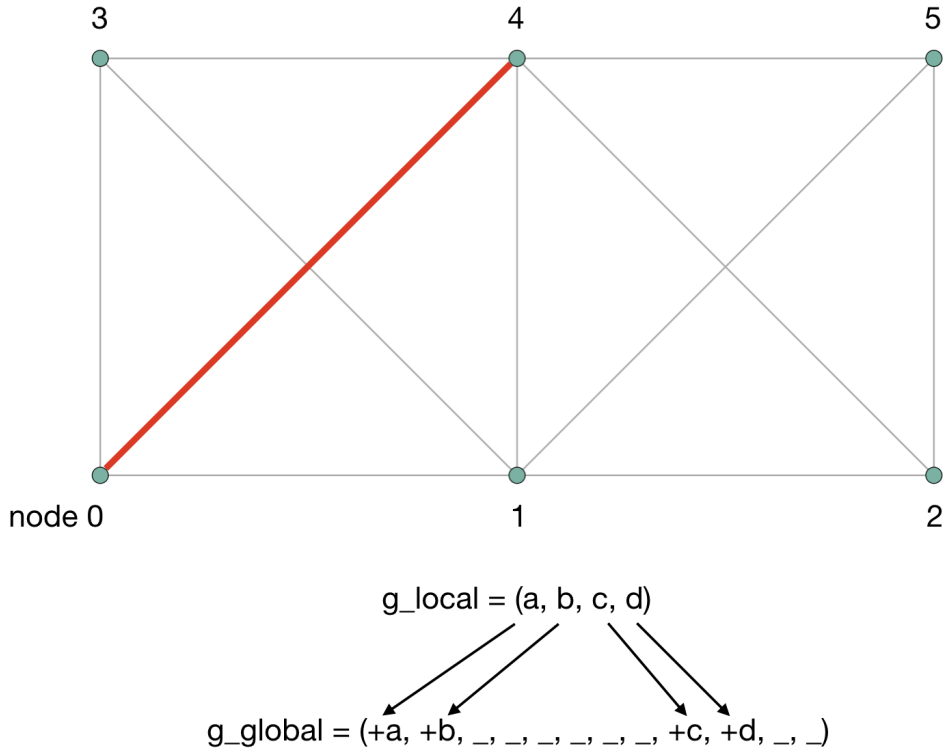


Figure 2: Example: assemble local gradient into the global gradient vector

There are two implementations of the `MassSpringProblem`, one with dense hessian matrix and the other with sparse hessian matrix. You should implement the corresponding functions in the two files `MassSpringProblem2DDense.hh` and `MassSpringProblem2DSparse.hh`. Different from setting up the eigen dense matrix, we need to use the triplets to create the sparse matrix. By comparing the performance of these two versions, we will see the advantage of the sparsity.

For implementation, the first step is to implement the `setup_spring_graph()` function in `MassSpringSystemT_impl.hh` to construct the spring graph. You can make use of the data structure provided in `MassSpringSystem/SpringGraph.hh`. Make sure the spring graph is correctly setup. In the executable, you can choose to output the spring graph as two files (.csv) and visualize them on the [website](#). Regarding the elastic constant k and the length l , for simplicity all k is set to 1 and the diagonal edge length l is set to $\sqrt{2}$ and the rest is 1.

You can also run a series of unit tests by running the `MassSpringProblemEvaluation-test` executable that is built with the standard executable. Those tests allow you to

incrementally verify that the smaller pieces of your implementation work as expected.

NOTE: passing all tests does not guarantee the validity of your solution.